

Erarbeitung und Reflexion des Pinguinklassifikators

Sami Stephan · IU14112704 · DLMDSPMLSD01_D

Auf Grundlage meines Konzepts habe ich den Pinguinklassifikator mit Python und Dash als lokale Browseranwendung umgesetzt. Vier Körpermesswerte und das Geschlecht dienen zur Unterscheidung von Adelie, Chinstrap und Gentoo; die Insel wird nur als Kontext gespeichert. Nach dem Ausschluss unvollständiger Körpermessungen bleiben 342 Beobachtungen des Palmer-Penguins-Datensatzes. Fehlende Geschlechtsangaben werden als „unbekannt“ verarbeitet. Die Oberfläche verbindet Dateneingabe, Vorhersage, grafischen Vergleich und CSV-Speicherung.

Die Dash-Callbacks waren für mich nicht einfach umzusetzen. Eingaben, Ergebnisanzeige und Speicherfunktion mussten zuverlässig zusammenspielen. Ein Callback-Zyklus ließ sich durch die Trennung der Statusaktualisierung auflösen. Außerdem musste sichergestellt werden, dass nach einer Eingabeänderung keine veraltete Vorhersage gespeichert wird. Um diese Abläufe nachvollziehbar zu halten, habe ich Oberfläche, Ereignisverarbeitung, Modellservice und Speicherung getrennt und den Code kommentiert. Die README beschreibt Einrichtung und Bedienung.

Auch eine praktikable Lösung für das Re-Training erforderte viel Überlegung. Entscheidend war, welche neuen Beobachtungen dafür geeignet sind: Modellvorhersagen dürfen nicht ungeprüft als tatsächliche Art zurück ins Training fließen. Deshalb werden nur unabhängig fachlich bestätigte Arten verwendet; doppelte und widersprüchliche Einträge werden geprüft. Das Re-Training wird manuell gestartet. Die neue Modellversion wird zunächst separat bewertet und erst nach ausdrücklicher Übernahme aktiviert. Die vorherige Version bleibt wiederherstellbar.

Bei der Modellbewertung mit Accuracy, Macro-F1, Cohen's Kappa und Konfusionsmatrizen fiel mir die fehlerfreie Klassifikation der Trainingsdaten auf. Das weckte meinen Verdacht auf Overfitting und führte zu einer weiteren Prüfschleife. Ich verglich neun Random-Forest-Einstellungen in einer fünffachen stratifizierten Kreuzvalidierung. Für neue Trainingsläufe begrenzte ich die Baumtiefe auf 5. Dadurch sinkt die mittlere Blattzahl um rund 24 %, bei 98,07 % statt 98,54 % Macro-F1. Zusätzlich prüfte ich eine logistische Regression, die auf denselben Folds 98,48 % erreicht. Für die kleine Datenbasis ist sie damit eine ernsthafte, einfachere Alternative. Vorerst bleibt der begrenzte Random Forest vorgesehen; bei weiteren Daten sollte die Modellwahl erneut geprüft werden. Die 86 ursprünglichen Testfälle blieben aus diesen Vergleichen ausgeschlossen. Fehlerfreie Trainingswerte allein belegen allerdings kein Overfitting, und die Vergleiche liefern keinen neuen unabhängigen Gütenachweis.

Für die Bereitstellung habe ich Docker und Startskripte eingebunden. Ein protokollierter Container-Test bestätigte Speicherung, Re-Training, Modellwechsel und Datenerhalt nach dem Neuerstellen des Containers. Nach weiteren Überarbeitungen sind 164 automatisierte Tests als bestanden dokumentiert. Vor der Abgabe stehen die erneute Prüfung des aktualisierten Images, der Offlinebetrieb und der Abgleich mit dem GitHub-Repository aus.

Repository: <https://github.com/SamiSte/penguin-classifier-python>